🚗 Vehicle Booking System

A **Ride Booking framework** must create different types of vehicles:

- Car
- Bike
- Van

The framework has an abstract base class RideService that implements the overall workflow of booking a ride.

The base class declares **one overridable method** whose job is to return the Vehicle object to use.

There is:

- One common interface Vehicle
- Each concrete service subclass (CarService, BikeService, VanService) overrides that single method to provide the appropriate vehicle.
- To support a new vehicle (e.g., Bus), you add a new service subclass and vehicle class — no changes to client code.

➢ **Which design pattern is most appropriate here?**

➢ **Draw a UML diagram for your design?**

➢ **Write a simple Java code?**

Factory Method Question

Solution:

Product Interface

java

```java
interface Vehicle {
    void startRide();
}
```

Concrete Products

java

```java
class Car implements Vehicle {
    public void startRide() {
        System.out.println("Car ride started");
    }
}

class Bike implements Vehicle {
    public void startRide() {
        System.out.println("Bike ride started");
    }
}

class Van implements Vehicle {
    public void startRide() {
        System.out.println("Van ride started");
    }
}
```

Factory Method Question

Abstract Creator

```java
abstract class RideService {

    public void bookRide() {
        Vehicle vehicle = createVehicle();   // Factory Method
        vehicle.startRide();
    }

    protected abstract Vehicle createVehicle();
}
```

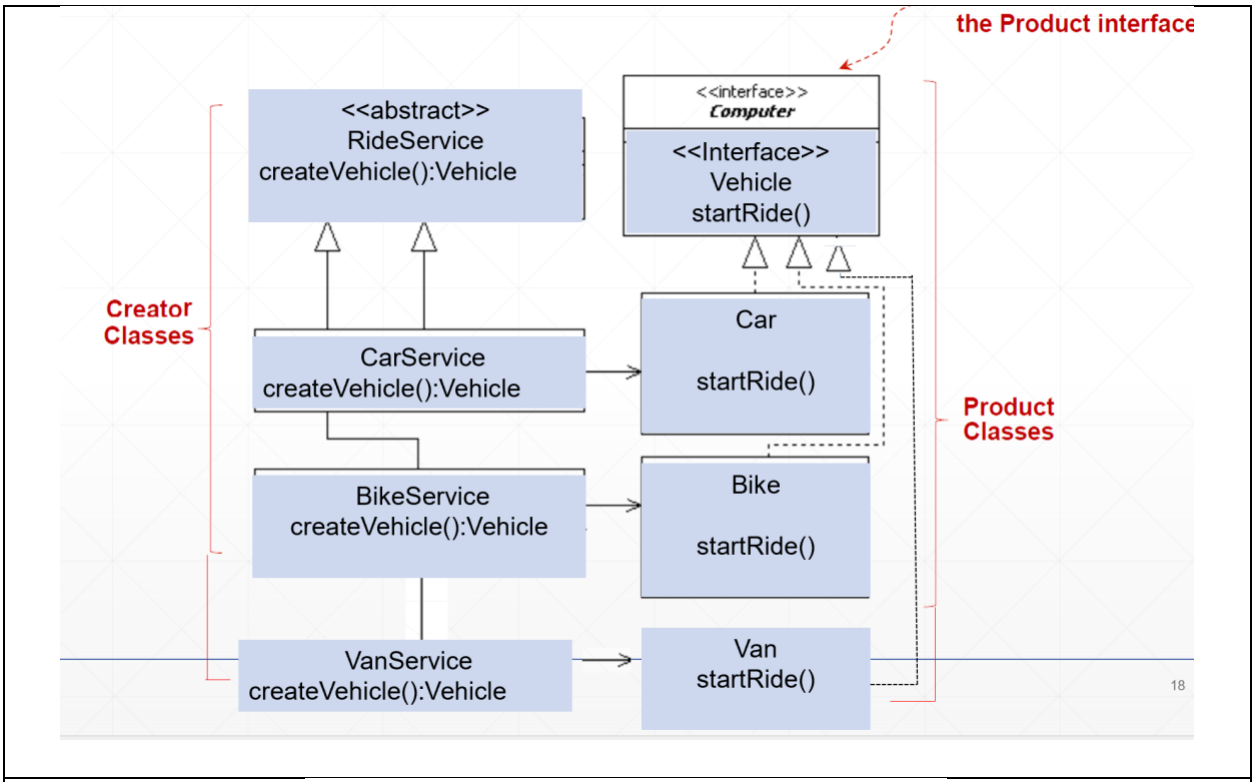
Concrete Creators

```java
class CarService extends RideService {
    protected Vehicle createVehicle() {
        return new Car();
    }
}

class BikeService extends RideService {
    protected Vehicle createVehicle() {
        return new Bike();
    }
}

class VanService extends RideService {
    protected Vehicle createVehicle() {
        return new Van();
    }
}
```

Factory Method Question



Client (Main)

```java
public class Demo {
    public static void main(String[] args) {

        RideService service1 = new CarService();
        service1.bookRide();

        RideService service2 = new BikeService();
        service2.bookRide();

        RideService service3 = new VanService();
        service3.bookRide();
    }
}
```